SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

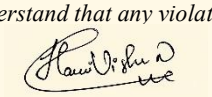
Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – SCENARIO BASED ASSIGNMENT

Course Code:	CSA17	**Course Title:**	Artificial Intelligence
CO Assessed:	CO2	**Assessment:**	Scenario Based Assignment
Weightage:	20% of CIA	**Duration / Marks:**	60 mins
CO2 Statement:	Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)		
Student Name:	HARI VISHNU N	**Reg. No.:**	192524319
Section / Batch:	SLOT D / 2025	**Date:**	21-10-2026

Code of Conduct

I HARI VISHNU N / 192524319 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: 

Instructions

- Read each scenario carefully before answering (Off-Class Assessment)
- Answer all five scenario-based questions. Each question carries and is designed to be answered in about 12 minutes.
- Identify the key issues in each scenario, apply the relevant AI technique, and justify every decision with clear reasoning.
- Questions are drawn from Informed Search (Greedy, A*), Local Search and Optimization, Online Search, Constraint Satisfaction Problems, and Adversarial Search (Minimax, Alpha-Beta, evaluation functions).
- Marks will be awarded using the Scenario Based Assignment rubric: understanding of scenario, identification of key issues, application of concepts, decision making and clarity of response.
- Upload your work in GCR and Github.

Section / Topic	Focus	Question Nos.
1. Informed Search Strategies	Greedy best-first search, A* search, heuristic functions	**Q1**
2. Local Search and Online Search	Local search algorithms and optimization problems (hill climbing, simulated annealing, genetic algorithms), online search agent	**Q2, Q3**
3. Constraint Satisfaction Problem	Variables, domains, constraints, backtracking search, constraint propagation (forward checking)	**Q4**
4. Game Playing: Adversarial Search	Games, minimax algorithm, alpha-beta pruning, evaluation functions	**Q5**

Question 1: Emergency Medicine Delivery Drones in a Flood Zone

Topic: Informed search strategies (Greedy best-first search, A, heuristic functions) | 12 minutes*

A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic: roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to:

- A heuristic estimate (straight-line distance) to the destination.
- Partial real-time updates about blocked paths.
- Variable movement costs depending on terrain and weather.

Due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

(a) Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty.

(b) Explain how A* would handle the same situation, including how it balances cost and heuristic.

(c) Analyze the impact of heuristic accuracy on both algorithms.

(d) Discuss how dynamic changes (blocked paths) affect search performance.

(e) Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety.

Answer 1:

The drone problem is a route-planning task in a dynamic, partially observable, and time-critical environment. Edge costs depend on weather and terrain, roads may be blocked during flight, and the drone must decide quickly while still arriving safely. The performance measures are delivery time, total risk, and the probability of arrival. The key issues are that the straight-line distance is only a rough guide to true cost, that the map is incomplete and changing, and that speed and safety must be balanced.

(a) Greedy Best-First Search

Greedy Best-First Search ranks each frontier node only by h(n), its straight-line distance to the hospital, and always expands the node that looks closest to the goal. Suppose a river lies between the depot and the hospital and the only crossing is a flooded bridge several kilometres upstream. The near-bank nodes look closer to the hospital, so greedy search explores them first, reaches a series of dead ends, and must then backtrack a long way. The algorithm is fast and simple, which suits an urgent departure. However, it ignores the path cost g(n), so weather-dependent costs and flooding are invisible to its decisions. It is neither optimal nor reliable here, and under uncertainty it commits early to a direction without weighing the risk.

(b) A* Search

A* ranks nodes by f(n) = g(n) + h(n), where g(n) is the weather-adjusted cost travelled so far and h(n) estimates the remaining cost. Consider two routes: route A is 12 km through open roads at a total cost of 12 units, and route B is 8 km through flooded districts at a total cost of 20 units. Greedy search may favour route B because it points more directly at the goal. A* compares f values, and the g term makes it prefer route A. For optimality, the heuristic must be admissible, so it never overestimates the true cost. A safe choice is h(n) = straight-line distance × the lowest cost per kilometre achievable under current conditions. A* balances cost and direction: when the map is open, h dominates and the search is fast, and when hazards exist, g dominates and the search goes around them. Its limitation is memory use on very large maps and its need to be rerun when the map changes.

(c) Impact of heuristic accuracy

Accurate heuristic. Greedy search moves almost directly to the goal, and A* expands few nodes beyond the optimal route.

Underestimating heuristic. Greedy search wastes effort. A* stays optimal because the heuristic is admissible, but it expands more nodes, so planning is slower but the route is still correct.

Overestimating heuristic. Greedy search is drawn into dead ends. A* may discard the optimal route and return a suboptimal one, which is unacceptable for medicine delivery.

Straight-line distance is inaccurate during storms, when true costs can be several times larger, so it underestimates. This is safe for A* but slows it down. Where speed matters more than strict optimality, weighted A* with f = g + w × h and w = 1.2 keeps the route within 20 percent of optimal while planning much faster.

(d) Effect of dynamic changes

A blocked road invalidates every planned route through it. Re-running A* from scratch is correct but slow, and the drone may have to hover during the delay. Greedy search suffers most, because its frontier is biased toward the goal and holds few alternatives around the new obstacle. Incremental algorithms such as D* Lite are designed for this case. They search backward from the goal, keep earlier cost estimates,

and update only the affected nodes, so a new route can be found quickly. Because updates arrive partially, the drone should plan optimistically, treating unknown roads as passable with a risk penalty, and replan whenever a sensor update arrives.

(e) Recommendation

I recommend A* combined with incremental replanning (D* Lite) as the primary method, with weighted A* as a fallback when computation is tightly limited. The choice is justified against the three criteria in the scenario.

Real-time constraints. D* Lite reuses earlier computation, so replanning after each update is much faster than a full search. Weighted A* gives a quick first route when time is very short.

Optimality. A* with an admissible heuristic guarantees the cheapest route under the current map. A longer or riskier route consumes battery and exposes the drone to hazards.

Safety. Safety comes from the cost function. Poorly observed areas carry a penalty, so the planner prefers well-known corridors even if they are slightly longer. Greedy search cannot express such preferences in a principled way.

The overall design is layered: D* Lite for continuous replanning, A* for the underlying optimal route, and a conservative risk-adjusted cost function for safety. Its limitations are reliance on accurate weather models and on network updates, so a fallback procedure for operating with limited communication is needed.

Worked comparison on the same flood-zone map

To make the difference concrete, imagine the drone at a junction with three options. Option X is a short road with a 60 percent chance of flooding, option Y is a longer road that is well observed and open, and option Z is a medium-length road with no recent data. Greedy search would pick whichever option has the smallest straight-line estimate to the hospital, which could be option X even though it is risky. A* would compute the cost so far and the optimistic remaining cost for each option; if the risk is represented as an expected extra cost, option X's f value rises and option Y becomes competitive. Option Z is not chosen merely for being unknown, because the planner assigns it an exploration penalty that reflects missing information. The result is a route chosen on cost and risk, not direction alone.

Handling uncertainty in the cost model

The most important design decision for this drone is how uncertainty enters the cost function. A reasonable approach is to set the cost of an edge as its base travel time, plus a weather multiplier, plus a risk term proportional to the probability that the edge is blocked multiplied by a large penalty. The penalty is set high enough that the planner avoids any road with significant blockage risk unless no alternative exists. This turns an uncertain environment into a deterministic search problem that A* and D* Lite can solve, while keeping the safety preferences explicit and auditable by the government team.

Computational budget and onboard constraints

A drone has limited processing power and battery. A full A* search over a city-sized road graph may take longer than the planning window if the map is large, so the design should partition the map into regions and plan hierarchically. A coarse route is planned across regions first, and detailed routing is computed only within the next few regions. Because D* Lite updates only affected nodes, the computational cost of each replanning step depends on the size of the change rather than the size of the whole map. This keeps the drone responsive, which is essential when new blockage reports arrive every few minutes.

Justification of the recommendation

The recommendation is justified because it treats the three requirements of the scenario as constraints on one design rather than as competing goals. Speed is achieved through incremental replanning rather than repeated full searches. Optimality is achieved through an admissible, weather-aware cost model. Safety is achieved by penalising uncertain roads and by refusing to commit to a route whose blockage risk is high. A purely greedy strategy would fail the safety requirement, and a purely exhaustive strategy would fail the timing requirement, so the chosen combination is the only one that meets all three.

Summary of the reasoning

Greedy best-first search is fast but blind to cost, so it is useful only as a guide for the first heading. A* uses the true cost of the path so far together with an admissible heuristic, and therefore produces routes that are cheapest under the current information. D* Lite adapts those routes efficiently as blockages change. Together with a risk-aware cost function and a fallback to weighted A* when computation is scarce, this combination answers every requirement in the scenario: rapid decisions, minimal delay, controlled risk, and safe arrival.

Question 2: Smart City Traffic Signal Optimization

Topic: Local search algorithms and optimization problems (hill climbing, simulated annealing, genetic algorithms) | 12 minutes

A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize:

- Total waiting time.
- Fuel consumption.
- Traffic congestion.

The system must continuously adjust signals based on traffic flow. However, the search space is extremely large, traffic patterns change dynamically, and the system may get stuck in locally optimal solutions.

Tasks:

(a) Formulate this as an optimization problem and explain why traditional search is inefficient.

(b) Explain how Hill Climbing would work in this scenario and identify its limitations.

(c) Discuss issues like local maxima, plateaus, and ridges with real-world interpretation.

(d) Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations.

(e) Recommend the best approach and justify your choice based on scalability and adaptability.

Answer 2:

The traffic problem is an optimisation problem rather than a path-finding problem. The goal is a good configuration of signal timings across the whole network, not a route from a start state to a goal state. The main difficulties are the very large number of intersections, the interaction between neighbouring junctions, traffic that changes over time, and an objective that can only be measured by observation or simulation. These features motivate local search methods.

(a) Formulation and why traditional search is inefficient

A system state is the complete set of control parameters (green split, cycle length, and offset) for all N intersections. The objective is to minimise f = w1 × waiting time + w2 × fuel consumption + w3 × congestion, subject to constraints such as minimum pedestrian-safe green times, bounded cycle lengths, and offsets that keep coordinated corridors consistent. If each intersection has 20 candidate timings and there are 300 intersections, the state space contains 20^300 configurations, far too many to enumerate. Traditional search also assumes a known transition graph and a recognisable goal. Here there is no single correct configuration, the effect of a change depends on fluctuating demand, and each evaluation is expensive. Local search is suitable because it moves between complete configurations, evaluates neighbours, and needs little memory.

(b) Hill Climbing

Hill climbing starts from the active timing plan and proceeds as follows:

1. Measure traffic and compute f for the current configuration.

2. Generate neighbours by changing one parameter at one intersection, such as increasing the northbound green at junction 17 by 5 seconds.

3. Accept the neighbour that gives the largest improvement (or the first improvement found).

4. Repeat until no neighbour improves f, then keep the current plan.

This is fast, uses little memory, and responds continuously to changing traffic, which suits daily operation. However, it reaches only a local optimum, depends heavily on its starting plan, and cannot cross plateaus or narrow ridges. A poor plan left over from the previous peak may persist because no single change looks beneficial.

(c) Local optima, plateaus, and ridges

Local optimum. Every single-junction change worsens delay, yet a coordinated green wave along the main road would clear the queues. Each junction is tuned to its own side street, so no single adjustment helps.

Plateau. At night, when traffic is light, many plans give almost the same delay. The search receives no gradient and wanders or stops early.

Ridge. Adjusting junction 18 helps only if junction 17 has already released platoons at the right time. Changing either alone worsens the objective, so hill climbing stalls on the side of the ridge. This is the classic arterial coordination problem.

(d) Simulated Annealing and Genetic Algorithms

Simulated annealing accepts an improving neighbour always, and a worsening neighbour with probability exp(−ΔE / T). The temperature T starts high, so the controller explores broadly and can leave a poor coordination pattern, and it is slowly reduced until the controller settles into a good configuration. This

addresses local optima directly and allows drifting across plateaus. A genetic algorithm maintains a population of timing plans, uses fitness-based selection, combines corridor segments through crossover, and applies small mutations. Crossover can transfer an entire coordinated corridor in one step, which is the diagonal move a ridge requires, and the population keeps several regions of the landscape alive at once. Both methods have costs: annealing needs a tuned cooling schedule, and genetic algorithms need careful encoding and a repair step for infeasible plans. Both are too slow for second-by-second control but suit periodic optimisation.

(e) Recommendation

I recommend a hierarchical hybrid, with each method used where it performs best, judged by scalability and adaptability.

Junction control (hill climbing). Each junction or district makes small, fast adjustments to green times. This scales because districts operate independently and react immediately to local change.

Network coordination (simulated annealing). Every 15 minutes, or when a significant pattern change is detected, annealing optimises corridor offsets across the network. Reheating provides adaptability, and the run finishes within the update interval.

Pattern library (genetic algorithm). Overnight, a genetic algorithm builds coordinated plans for common profiles such as morning peak, evening peak, and event days. These are validated in simulation and stored for rapid switching.

Pure hill climbing would miss coordination gains, pure annealing across the whole city would be too slow for real-time control, and pure genetic algorithms would be too expensive to run continuously. The hybrid keeps the speed of local control and the global quality of population search. Minimum pedestrian crossing times and emergency-vehicle priority must remain hard constraints, and a fixed safe plan should be used if sensor data becomes unreliable.

Worked example of a local optimum being escaped

Consider a corridor of four intersections, A to D, where the main road is the priority direction. A hill-climbing controller finds that changing any single junction increases delay, because each junction's side street would then queue. The controller therefore stops at a configuration with an average delay of 90 seconds per vehicle. Simulated annealing, at its initial temperature, may accept a move that worsens delay to 105 seconds by lengthening the cross-street green at junction B and shortening the main-road green there. This temporary worsening lets the search reach a region in which the offsets at A, B, C, and D can be coordinated. As the temperature falls, the controller keeps the improved pattern, and the corridor settles at around 70 seconds. The example shows why accepting temporary losses is the key to escaping a trap that greedy improvement cannot leave.

Choosing the cooling schedule

The performance of simulated annealing depends mainly on the cooling schedule. If the temperature falls too quickly, the search behaves like hill climbing and gets trapped; if it falls too slowly, the computation may exceed the update interval. A geometric schedule, in which the temperature is multiplied by a constant factor such as 0.95 after each batch of moves, is a practical choice. The number of moves per temperature should be tied to the number of intersections, so that every junction is considered at each level. Restarting the schedule at a moderate temperature when a traffic change is detected allows the controller to adapt without starting from scratch.

Design of the genetic algorithm

For the offline pattern library, each chromosome encodes a timing plan as a list of green splits and offsets, one segment per corridor. The fitness of a chromosome is the negative of the simulated objective value, evaluated over a representative demand profile. Tournament selection chooses parents, uniform or segment-wise crossover exchanges whole corridor blocks, and a small mutation rate adjusts individual green times. After each generation, a repair operator enforces minimum green times and cycle limits. Elitism keeps the best plan in each generation, so the library's quality never decreases. These design choices allow the algorithm to discover coordinated corridor patterns that hill climbing cannot reach.

Comparison of the three methods for the city

Hill climbing offers the fastest response and the smallest memory footprint, but it can only improve plans locally. Simulated annealing needs more computation per decision and must be scheduled carefully, but it escapes local optima and handles network-wide coordination. The genetic algorithm produces the highest-quality patterns, but its evaluation cost is high and it cannot be run every few minutes across a whole city. Judged on scalability, hill climbing scales best because its work is local; judged on adaptability, annealing with reheating is strongest for changing traffic; judged on solution quality over recurring patterns, the genetic algorithm is best. The hybrid is designed to use each method for the criterion where it leads.

Limitations and governance

Optimised timings depend on the quality of the traffic sensors and the accuracy of the simulator used for offline evaluation. A stored pattern that performs well in simulation may still fail during an unusual event, so the system should monitor performance after each change and roll back to the previous plan if congestion rises beyond a threshold. Transparency is also important, because drivers and local authorities should be able to see why a timing was changed. Logging the objective values before and after each change supports this requirement and helps engineers refine the weights in the objective function.

Question 3: Autonomous Rover Exploring Mars

Topic: Online search agent | 12 minutes

An autonomous rover is deployed on Mars to explore unknown terrain. It must:

- Navigate safely to collect samples.
- Avoid hazards (craters, rocks).
- Operate with limited sensing capability.
- Make decisions without a complete map.

The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

(a) Explain why this problem requires an online search agent instead of offline search.

(b) Analyze the challenges of partial observability and dynamic environments.

(c) Describe how the rover builds and updates its internal model.

(d) Compare online search with uninformed and informed search strategies.

(e) Justify the most suitable approach considering uncertainty, adaptability, and safety.

Answer 3:

The rover explores unmapped terrain with limited sensors, dynamic conditions, and no complete map. It must choose each action before knowing the full environment. This defines an online search problem, in which the agent interleaves planning and acting: it moves, observes, updates its knowledge, and decides again.

(a) Why an online search agent is required

Offline search, such as A* on a complete map, assumes that the full state space is known, that the outcome of every action is known, and that the environment does not change. None of these holds on Mars. The rover cannot know which cells are craters or rocks until its sensors reveal them, and the terrain may change during the mission. An online agent discovers the successors of a state only when it visits that state, so each move yields new information and the next decision uses the latest observation. Remote planning is also impractical because communication delays of several minutes prevent Earth from issuing step-by-step commands. The online approach is therefore the only realistic option.

(b) Partial observability and dynamic environments

Partial observability means the rover sees only nearby terrain, so a crater beyond the horizon is invisible until it is approached. The rover's knowledge is therefore a belief about the map rather than the map itself. The main challenges are as follows.

Unseen hazards. Treating unknown terrain as safe could lead the rover into a crater, so unknown cells must be handled cautiously.

Sensor noise. Dust and low light make readings unreliable, so repeated observations are needed before a cell is labelled hazardous.

Localisation error. Wheel slip on loose sand makes odometry drift, which misaligns the map.

Changing terrain. Dust storms, shifting sand, and slippage can make a previously safe route unsafe.

Because the map is never final, the rover must keep revising its beliefs, maintain safety margins, and stop to reassess whenever observations contradict its model.

(c) Building and updating the internal model

The rover maintains an occupancy grid in which each cell holds a hazard probability. Each sensor reading updates the probabilities of the cells in view using Bayes' rule, often in log-odds form. The rover estimates its position by combining odometry with landmark observations. The frontier, the set of known free cells bordering unknown cells, becomes the set of exploration targets. Planning is optimistic: unknown cells are assumed passable but carry an exploration cost and a risk penalty. The learning-based algorithm LRTA* further stores a heuristic estimate H(s) for each visited state and raises it to reflect experienced costs, so repeated trips through the same region become more efficient. An occupancy grid is transparent, fits the sensing model, and supports probabilistic updates.

(d) Comparison with uninformed and informed search

Breadth-first and depth-first (uninformed). Both require a complete map. Breadth-first search needs too much memory for a rover, and depth-first search can wander far from the lander. Neither responds to new observations without restarting.

A* (informed). A* produces optimal paths and uses a heuristic to focus the search, but it needs a known graph and must be rerun whenever the map changes.

Online search (LRTA*, D* Lite). These interleave planning and action, work with partial information, and replan incrementally when hazards appear. Their paths may be longer than the optimal offline path, a difference often measured by the competitive ratio. This is acceptable when adaptability and safety matter more than path length.

(e) Justified selection

I recommend an online search agent using LRTA* or D* Lite on a probabilistic occupancy grid, with frontier-based exploration and an independent safety layer.

Uncertainty. The probabilistic grid represents uncertainty explicitly, and the planner balances information gain against hazard risk.

Adaptability. Incremental replanning updates the route when a new hazard appears, and LRTA* improves its estimates from experience.

Safety. A separate safety layer enforces margins around hazards, blocks moves into cells above a risk threshold, and forces a stop when sensor data contradicts the model. A planning error therefore cannot override safety.

The rover should prefer frontier cells that are close and low-risk, and should keep a route back to the lander. If localisation is lost, it should stop and re-establish its position from landmarks before continuing. Its main cost is a longer explored path and greater onboard computation, which is an acceptable trade-off for a mission where safety and reliability come first.

Worked example of online exploration

Consider a rover that starts at a lander and wants to reach a sample site 30 metres away. Its initial map shows only the cells within 5 metres. The planner treats unknown cells as passable with an exploration cost, so it heads toward the sample site along the straight line. After two metres, a camera observation reveals a dark patch that is probably a crater edge. The occupancy grid raises the hazard probability of those cells, and the planner immediately recomputes the route, sidestepping to the left. Had the rover followed a plan computed before the observation, it would have driven into the crater. The example shows that the value of the online approach lies in the frequent, cheap updates it makes possible, not in the optimality of any single plan.

Safety margins and stopping rules

Because a hazard may be only partly visible, the rover should keep a safety margin around every cell whose hazard probability is above a low threshold, and it should treat the margin as an obstacle for planning. Stopping rules are equally important. If the rover's localisation estimate changes by more than a set tolerance in a single step, or if a sensor reading contradicts several recent observations, the rover should halt and wait for a further reading or a command from Earth. These simple rules protect the hardware against the most dangerous failures, which are usually caused by a model that has become inconsistent with the terrain.

Choosing between LRTA* and D* Lite

LRTA* is attractive for repeated traversal of a region, because its learned heuristic improves each time the rover visits a state. Its disadvantage is that early trips can be inefficient while the estimates are still poor. D* Lite is attractive when the environment changes after the initial exploration, because it updates the costs of affected cells incrementally without discarding the earlier search. For a rover that explores once, must return to the lander, and encounters new hazards, a combination is sensible: D* Lite provides

incremental replanning during each sortie, and learned estimates from LRTA* reduce the cost of returning along known routes.

Energy and communication considerations

Every manoeuvre consumes energy, and the rover cannot recharge quickly on Mars. The cost function should therefore include an energy term that penalises steep slopes and loose sand, so that the planner favours routes that are cheap to traverse, not just short. Communication delays also mean that Earth cannot act as a live planner. The rover should send periodic summaries of its occupancy map so that scientists can review its progress, but all time-critical decisions must be made onboard. This division of responsibility reflects the reality of interplanetary operations.

Summary of the recommendation

Offline search cannot be used because the map does not exist in advance, and uninformed search cannot be used because it cannot exploit the sensor information that the rover receives. Informed offline search is appropriate only after the map has been built. An online agent that combines a probabilistic occupancy grid, optimistic exploration, incremental replanning, and an independent safety layer offers the best combination of uncertainty handling, adaptability, and safety for this mission, with the trade-off of longer exploratory paths.

Question 4: University Exam Timetable Generation

Topic: Constraint satisfaction problem (backtracking search, constraint propagation, forward checking) | 12 minutes

A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints:

- No student should have overlapping exams.
- Limited number of exam halls.
- Certain subjects must be scheduled before others.
- Faculty availability constraints.
- Special accommodations for some students.

The complexity increases as the number of courses and students grows.

Tasks:

(a) Formulate the problem as a Constraint Satisfaction Problem (CSP): clearly define the variables, domains, and constraints with explanation.

(b) Explain how backtracking search works in this scenario.

(c) Discuss how constraint propagation (for example, forward checking) improves efficiency.

(d) Analyze real-world challenges and justify the best strategy for scalability.

Answer 4:

Timetabling is a classic constraint satisfaction problem (CSP). The aim is not to minimise a path cost but to find any assignment of slots and halls that satisfies many interacting rules. The number of candidate timetables grows rapidly with the number of courses, so the method must use inference and good ordering to avoid hopeless combinations.

(a) CSP formulation

Variables. One variable X_c for each course or exam paper that must be scheduled.

Domains. Each variable takes a value (t, h), where t is a time slot and h is an examination hall. With 25 slots and 8 halls, each domain starts with 200 values.

Conflict constraints (binary). Two courses sharing at least one student must not share a slot: time(X_c1) ≠ time(X_c2). These are the most numerous constraints.

Capacity constraints. No more exams may run in a slot than there are halls, and each hall must be large enough for its exam's enrolment. The hall-size rule is unary and removes unsuitable halls from a domain.

Precedence constraints (binary). If subject A must precede subject B, time(X_A) < time(X_B).

Faculty constraints. Unavailable slots are removed from an invigilator's courses' domains, and no invigilator may supervise two exams in the same slot.

Accommodations (unary). Students needing extra time or special rooms restrict their courses to suitable halls and longer slots.

This explicit model lets any violated rule be traced to a specific constraint, supports domain reduction, and keeps the university's rules separate from the solver so they can change without rewriting the algorithm.

(b) Backtracking search

Backtracking assigns one course at a time and checks constraints after each assignment. If a conflict arises, or an unassigned course is left with no legal value, the algorithm undoes the latest assignment and tries another. For example, suppose courses A, B, and C share students pairwise and only two slots exist. Assigning A to S1 and B to S2 is consistent, but C then has no legal slot. The search backtracks through B and A and eventually concludes that no solution exists. This shows both the correctness and the weakness of plain backtracking: failure is discovered only after several assignments. Three heuristics make it practical. Minimum remaining values (MRV) assigns the most constrained course first. The degree heuristic breaks ties by choosing the course with the most constraints on unassigned courses. The least constraining value (LCV) heuristic chooses the slot that keeps the most options for neighbours. Large courses with many student conflicts and tight faculty limits should be scheduled first.

(c) Constraint propagation and forward checking

Forward checking removes, after each assignment, any value that is now inconsistent from the domains of unassigned neighbouring courses. If any domain becomes empty, the algorithm backtracks immediately without exploring the subtree below. In the example above, assigning A to S1 removes S1 from the domains of B and C. Their domains then contain only S2, so the conflict between them is detected at once. This prunes the search tree early and reduces wasted work at a small cost per step. Arc consistency (AC-3) and maintaining arc consistency (MAC) propagate further by checking every constrained pair and the

effects of each removal. They prune more strongly but cost more per step, so MAC is worth using for tightly constrained instances, and forward checking is a sensible default.

(d) Real-world challenges and the best scalable strategy

Soft constraints. Preferences, such as avoiding back-to-back exams or spreading invigilation evenly, cannot be hard rules without risking infeasibility, so they are modelled as penalties in an objective.

Late changes. Illness, room closures, and changed enrolments require local repair rather than regeneration, which would disrupt arrangements already communicated.

Scale and density. As courses grow, the conflict graph becomes denser and feasible timetables become rarer, so exhaustive search becomes impractical.

Fairness and transparency. Students and staff expect explainable, unbiased results, so the system must be auditable.

A useful way to manage scale is to model student conflicts as a graph, where courses are vertices and shared students create edges. Scheduling then resembles graph colouring, and the degree of a course gives a useful bound for ordering. I recommend a two-stage strategy. First, backtracking with MRV, degree, LCV, and forward checking (or MAC for dense instances) produces a feasible timetable that satisfies all hard constraints. Second, min-conflicts local search repairs and improves the solution against soft constraints by repeatedly moving a conflicting course to the slot that minimises violations. The first stage guarantees validity, and the second scales efficiently and handles late changes without restarting. This combination is the most practical route to a scalable, reliable timetable.

Worked example of forward checking in a larger case

Suppose five courses, P, Q, R, S, and T, have the following conflicts: P conflicts with Q and R, Q conflicts with S, and R conflicts with T. There are three slots, one hall each for simplicity. Plain backtracking may assign P to slot 1, Q to slot 2, and S to slot 1, which violates no rule until it is checked, and then discover failures deep in the tree. With forward checking, assigning P to slot 1 immediately removes slot 1 from Q and R. If Q is then assigned slot 2, forward checking removes slot 2 from S, leaving S with only slot 3. Assigning R to slot 2 removes slot 2 from T, so T must take slot 3. The procedure reaches a complete, valid timetable with far less backtracking, because every assignment immediately narrows the choices of the courses it affects.

Handling hall capacity and invigilators

Hall capacity and invigilator availability are resource constraints that behave differently from student conflicts. A hall can host only one exam per slot, and an invigilator can supervise only one exam at a time, so these constraints resemble the all-different rule applied to resources. A practical approach is to treat each hall and each invigilator as a resource with a calendar, and to remove a slot from a course's domain as soon as the resource it needs is fully booked. This keeps the forward-checking step cheap, because the check reduces to a lookup in the resource calendar rather than an examination of every other course.

Ordering the exams for the best results

The order in which exams are assigned has a large effect on search time. Courses with very large enrolments need large halls, which are scarce, so they should be placed first. Courses with many student conflicts restrict the most neighbours, so the degree heuristic should bring them forward. Courses restricted to a small set of slots because of faculty unavailability or accommodation needs should also

come early, since their domains are small. Placing these courses first means that the scarce resources are allocated before flexible courses fill them, which reduces late failures.

Evaluating the strategy

The chosen strategy is evaluated on four criteria. Correctness is ensured because backtracking explores every possibility, so a valid timetable is found whenever one exists. Efficiency is improved by MRV, degree, and forward checking, which cut the number of nodes explored by orders of magnitude on typical timetabling instances. Adaptability is supplied by the min-conflicts phase, which can repair a small number of changes in seconds. Transparency is supported because each violated constraint can be reported to the timetabling office with the courses and students involved. Together these criteria make the approach suitable for a university that needs reliable and explainable results.

Practical considerations for deployment

Before deployment, the timetabling office should validate the constraint data, because most failures in practice come from incorrect enrolment lists or faculty availability records rather than from the solver. The system should also allow human planners to fix particular exams manually, which the solver then treats as fixed variables. This hybrid of human judgement and automated search is often more effective than either alone, and it addresses the fairness and trust concerns that automated timetables frequently raise.

Question 5: Strategic Game AI for Real-Time Decision Making

Topic: Game playing: adversarial search (minimax, alpha-beta pruning, evaluation functions) | 12 minutes

A gaming company is developing an AI opponent for a real-time strategy game. The AI must:

- Compete against human players.
- Make decisions within strict time limits.
- Evaluate complex game states with multiple possible moves.

The decision tree is extremely large, making computation expensive.

Tasks:

(a) Explain how the Minimax algorithm models this problem.

(b) Analyze the computational challenges of large game trees.

(c) Explain how Alpha-Beta pruning improves efficiency with detailed reasoning.

(d) Design an evaluation function and justify the factors considered.

(e) Recommend a strategy for balancing optimality and real-time performance.

Answer 5:

The AI plays a two-player, zero-sum game, so adversarial search is appropriate. Real-time strategy games differ from classical games because they have very large move sets, hidden information, and strict per-decision time limits. These differences shape how minimax and alpha-beta must be applied and make the evaluation function central.

(a) How Minimax models the problem

Minimax represents the game as a tree. The AI is the MAX player, which chooses the child with the highest value, and the opponent is the MIN player, which chooses the child with the lowest value. Terminal states have known utilities, and every other node's value is the maximum or minimum of its children's values. The AI therefore selects the move that performs best against the opponent's best reply. In a real-time game the full tree cannot be searched, so the search is cut off at a fixed depth and leaves are scored by an evaluation function. A simple case: a MAX node with two MIN children, the first with leaves 3 and 5 (value 3) and the second with leaves 2 and 9 (value 2). The AI chooses the first move, whose guaranteed value is 3.

(b) Computational challenges of large game trees

Minimax to depth d with branching factor b examines about b^d positions. A branching factor of 30 and depth 4 already require about 810,000 evaluations. In a real-time strategy game with dozens or hundreds of units, the number of joint actions grows so quickly that the branching factor can reach the thousands, and even depth 2 may exceed a budget of tens of milliseconds. Additional difficulties include memory consumption, probabilistic outcomes such as attacks that may fail, and hidden information, which forces the AI to reason over beliefs rather than a known state. Exact search is therefore impractical, and depth limits, pruning, move ordering, and abstraction are required.

(c) Alpha-Beta pruning

Alpha-beta keeps two bounds: alpha, the best value the MAX player is already guaranteed, and beta, the best value the MIN player is already guaranteed. At a MIN node, once a child value is at or below alpha, the remaining children cannot influence the MAX ancestor and are skipped. At a MAX node, once a value reaches beta, the rest are skipped. Using the example above, after the first MIN child gives alpha = 3, the second MIN child examines leaf 2 first. Its value can be at most 2, which is below 3, so leaf 9 is never examined. The decision is identical to minimax, with fewer nodes visited. With good move ordering, the number of nodes falls from about b^d to about b^(d/2), which allows roughly twice the search depth in the same time. Move-ordering techniques include searching the previous iteration's best move first, killer moves that caused cut-offs in sibling nodes, and history heuristics. Without good ordering, alpha-beta gains little, so ordering is essential.

(d) Evaluation function design

I propose a weighted linear evaluation: $\text{Eval}(s) = \Sigma\, w_i \times f_i(s)$, where each feature is normalised to a fixed range so that no single factor dominates. The features and their justifications are:

Resource difference. Gold, energy, or supplies determine future production, so they predict future strength.

Army strength. A sum of unit values weighted by type, health, and counter-relationships. This captures the immediate military balance.

Map control. The share of the map under the AI's influence, including key resource points and chokepoints, gives lasting positional advantage.

Technology progress. Completed upgrades provide advantages that outlast single battles, so they receive a moderate weight.

Threat level. Own units under attack, weighted by value, minus enemy units able to reach key structures soon. This captures tactical urgency that static features miss.

Economic growth. The income rate relative to the opponent, which can matter more than the current stockpile.

Weights can be tuned by self-play or by regression on recorded outcomes, and they should vary by game phase: economy matters more early, while army strength and threat dominate later. Positions in the middle of an exchange are unstable, so a quiescence search extends them until they settle, avoiding the horizon effect. For example, if the AI has a stronger economy but its army is under heavy attack, the threat term drops the evaluation and the AI prioritises defence.

(e) Balancing optimality and real-time performance

Exact optimality is unattainable within the time limit, so the aim is a strong move on time. The recommended strategy combines several techniques.

Iterative deepening with a time limit. The search proceeds to depth 1, 2, 3, and so on, and returns the best move from the deepest completed search when time expires. This is an anytime method that always responds on time.

Alpha-beta with strong move ordering. Previous best moves, killer moves, and history heuristics maximise pruning and allow greater depth.

Transposition table. Stored evaluations of previously seen positions avoid repeated work when different move orders lead to the same state.

Action abstraction. Grouping unit actions into strategic macro-moves reduces the branching factor, with tactical detail handled by a faster routine.

Safe fallback. A default safe move is returned if the search cannot complete, so the AI never stalls.

Difficulty levels can be created by limiting search depth or introducing occasional sub-optimal moves, which also makes the AI more human-like. This design accepts a near-optimal move in exchange for a guaranteed response, which is exactly what a real-time game requires.

Worked example of the search budget

Suppose the AI has 100 milliseconds per decision and each evaluation takes 20 microseconds. This allows about 5,000 leaf evaluations. With a branching factor of 30, plain minimax can reach only depth 2 or 3 within that budget. Alpha-beta with good move ordering can reach roughly depth 4 or 5 with the same number of evaluations, because it avoids examining many branches. Iterative deepening uses the first few shallow searches to order moves for the deeper search, so the total cost is only modestly larger than a single deep search. This is why the combination of these techniques is so effective in a timed game.

Calibrating the evaluation weights

The weights in the evaluation function should be calibrated so that the features are on comparable scales. For example, a unit of resources and a unit of army strength should not be treated as equal merely because both are numbers. A practical method is to normalise each feature to the range from minus one to plus

one, using the typical maximum observed in self-play, and then to assign weights from regression on game outcomes. The calibration should be repeated after each major game update, because changes to unit costs can shift the balance between features considerably.

Preventing the horizon effect

The horizon effect occurs when a damaging event lies just beyond the search depth, so the AI believes a position is safe when it is not. In a strategy game, a unit that will be destroyed in the next turn may appear healthy at the horizon. Quiescence search addresses this by continuing the search through captures and attacks until the position becomes stable. A threat-aware evaluation term also helps by penalising positions in which the AI's units are exposed, even if the immediate exchange has not yet occurred. Together these measures reduce the chance of an overconfident decision.

Human-like play and game design

The goal of an opponent AI is not only to win but to provide an enjoyable challenge. A perfectly optimal opponent may frustrate players, while a random one bores them. The depth limit, the weight profile, and an element of controlled randomness can be adjusted to produce a range of difficulty levels. Players also benefit when the AI appears to pursue sensible plans, such as expanding, defending when threatened, and attacking weak positions. The evaluation design should therefore be checked not only for winning strength but for the believability of the AI's behaviour.

Rubrics for Scenario Based Assignment

Criteria	Weightage	Level 4 – Exemplary (100% of marks)	Level 3 – Proficient (75%)	Level 2 – Developing (50%)	Level 1 – Beginning (25%)
Understanding of Scenario	**20%**	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	**20%**	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	**25%**	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	**20%**	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	**15%**	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q. No. / Criteria	Understanding of Scenario	Identification of Key Issues	Application of Concepts	Decision Making	Clarity of Response	Sub. Total	Total %
1							
2							
3							
4							
5							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____